

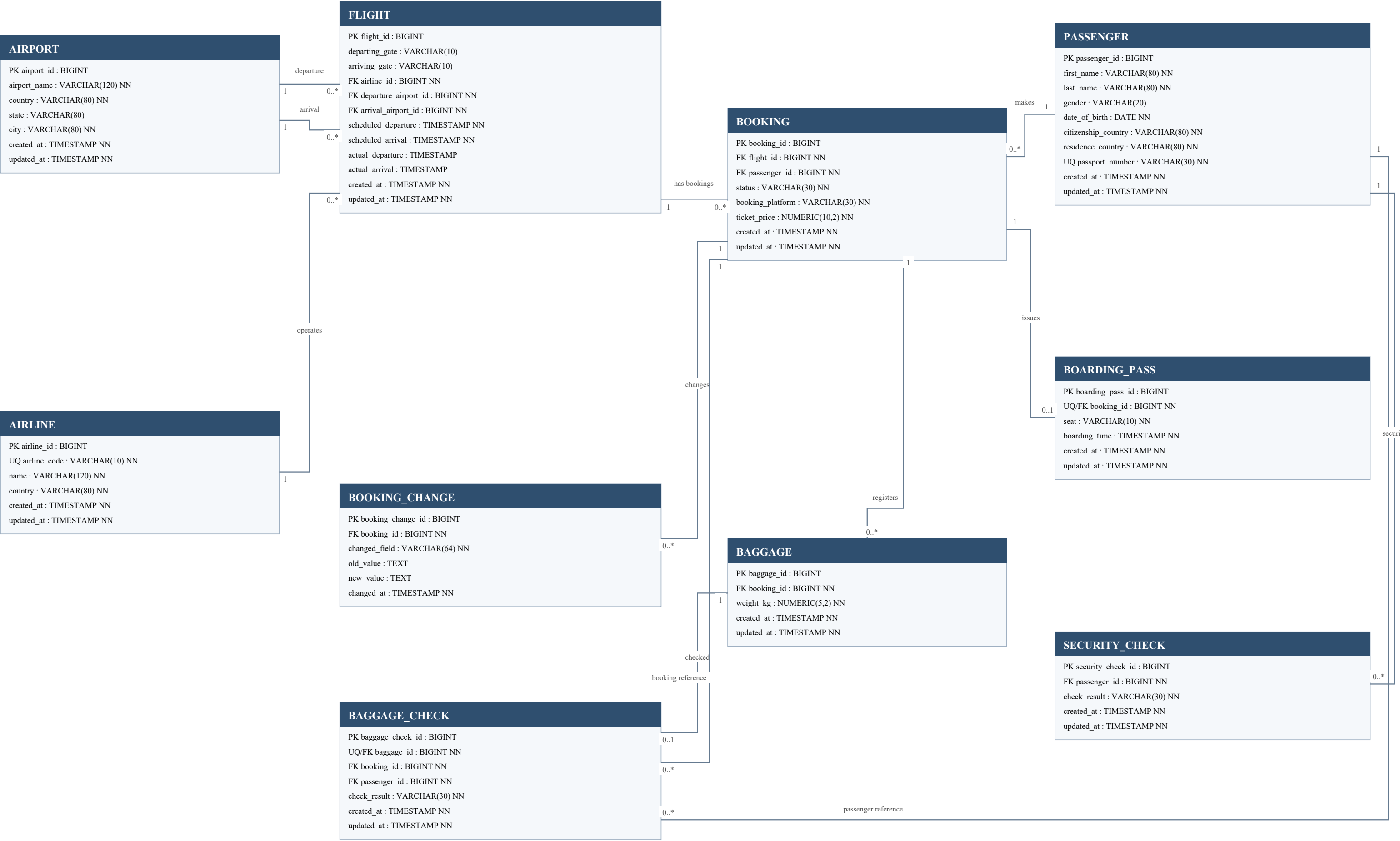
# **Laboratory Work 1**

ER Diagram for an International Airport Database

Student: Мадн Мурат

## 2 ER Diagram of the Airport Database

All attributes include a SQL data type. Cardinalities show minimum and maximum participation.



## 3 Entities and attribute data types - part 1 of 3

### AIRPORT

Stores the airports used as departure and arrival points for flights.

| Attribute    | Data type    | Keys and constraints    |
|--------------|--------------|-------------------------|
| airport_id   | BIGINT       | PK; generated; NOT NULL |
| airport_name | VARCHAR(120) | NOT NULL                |
| country      | VARCHAR(80)  | NOT NULL                |
| state        | VARCHAR(80)  | NULL allowed            |
| city         | VARCHAR(80)  | NOT NULL                |
| created_at   | TIMESTAMP    | NOT NULL                |
| updated_at   | TIMESTAMP    | NOT NULL                |

### AIRLINE

Stores airline companies that operate flights.

| Attribute    | Data type    | Keys and constraints    |
|--------------|--------------|-------------------------|
| airline_id   | BIGINT       | PK; generated; NOT NULL |
| airline_code | VARCHAR(10)  | UNIQUE; NOT NULL        |
| name         | VARCHAR(120) | NOT NULL                |
| country      | VARCHAR(80)  | NOT NULL                |
| created_at   | TIMESTAMP    | NOT NULL                |
| updated_at   | TIMESTAMP    | NOT NULL                |

### FLIGHT

Stores the schedule, actual times, gates, airline, and both airports for each flight.

| Attribute            | Data type   | Keys and constraints               |
|----------------------|-------------|------------------------------------|
| flight_id            | BIGINT      | PK; generated; NOT NULL            |
| departing_gate       | VARCHAR(10) | NULL until assigned                |
| arriving_gate        | VARCHAR(10) | NULL until assigned                |
| airline_id           | BIGINT      | FK -> AIRLINE.airline_id; NOT NULL |
| departure_airport_id | BIGINT      | FK -> AIRPORT.airport_id; NOT NULL |
| arrival_airport_id   | BIGINT      | FK -> AIRPORT.airport_id; NOT NULL |
| scheduled_departure  | TIMESTAMP   | NOT NULL                           |
| scheduled_arrival    | TIMESTAMP   | NOT NULL                           |
| actual_departure     | TIMESTAMP   | NULL until departure               |
| actual_arrival       | TIMESTAMP   | NULL until arrival                 |
| created_at           | TIMESTAMP   | NOT NULL                           |
| updated_at           | TIMESTAMP   | NOT NULL                           |

## 3 Entities and attribute data types - part 2 of 3

### PASSENGER

Stores the identity and travel-document information of passengers.

| Attribute           | Data type   | Keys and constraints    |
|---------------------|-------------|-------------------------|
| passenger_id        | BIGINT      | PK; generated; NOT NULL |
| first_name          | VARCHAR(80) | NOT NULL                |
| last_name           | VARCHAR(80) | NOT NULL                |
| gender              | VARCHAR(20) | NULL allowed            |
| date_of_birth       | DATE        | NOT NULL                |
| citizenship_country | VARCHAR(80) | NOT NULL                |
| residence_country   | VARCHAR(80) | NOT NULL                |
| passport_number     | VARCHAR(30) | UNIQUE; NOT NULL        |
| created_at          | TIMESTAMP   | NOT NULL                |
| updated_at          | TIMESTAMP   | NOT NULL                |

### BOOKING

Links one passenger to one flight and stores the booking details.

| Attribute        | Data type     | Keys and constraints                   |
|------------------|---------------|----------------------------------------|
| booking_id       | BIGINT        | PK; generated; NOT NULL                |
| flight_id        | BIGINT        | FK -> FLIGHT.flight_id; NOT NULL       |
| passenger_id     | BIGINT        | FK -> PASSENGER.passenger_id; NOT NULL |
| status           | VARCHAR(30)   | NOT NULL                               |
| booking_platform | VARCHAR(30)   | NOT NULL                               |
| ticket_price     | NUMERIC(10,2) | NOT NULL; value >= 0                   |
| created_at       | TIMESTAMP     | NOT NULL                               |
| updated_at       | TIMESTAMP     | NOT NULL                               |

### BOOKING\_CHANGE

Keeps an audit record whenever a booking field is changed.

| Attribute         | Data type   | Keys and constraints               |
|-------------------|-------------|------------------------------------|
| booking_change_id | BIGINT      | PK; generated; NOT NULL            |
| booking_id        | BIGINT      | FK -> BOOKING.booking_id; NOT NULL |
| changed_field     | VARCHAR(64) | NOT NULL                           |
| old_value         | TEXT        | NULL allowed                       |
| new_value         | TEXT        | NULL allowed                       |
| changed_at        | TIMESTAMP   | NOT NULL                           |

### 3 Entities and attribute data types - part 3 of 3

#### BOARDING\_PASS

Stores the boarding pass issued for a booking.

| Attribute        | Data type   | Keys and constraints                       |
|------------------|-------------|--------------------------------------------|
| boarding_pass_id | BIGINT      | PK; generated; NOT NULL                    |
| booking_id       | BIGINT      | FK -> BOOKING.booking_id; UNIQUE; NOT NULL |
| seat             | VARCHAR(10) | NOT NULL                                   |
| boarding_time    | TIMESTAMP   | NOT NULL                                   |
| created_at       | TIMESTAMP   | NOT NULL                                   |
| updated_at       | TIMESTAMP   | NOT NULL                                   |

#### BAGGAGE

Stores baggage registered for a booking.

| Attribute  | Data type    | Keys and constraints               |
|------------|--------------|------------------------------------|
| baggage_id | BIGINT       | PK; generated; NOT NULL            |
| booking_id | BIGINT       | FK -> BOOKING.booking_id; NOT NULL |
| weight_kg  | NUMERIC(5,2) | NOT NULL; value > 0                |
| created_at | TIMESTAMP    | NOT NULL                           |
| updated_at | TIMESTAMP    | NOT NULL                           |

#### BAGGAGE\_CHECK

Stores the check result for baggage and identifies its booking and passenger.

| Attribute        | Data type   | Keys and constraints                       |
|------------------|-------------|--------------------------------------------|
| baggage_check_id | BIGINT      | PK; generated; NOT NULL                    |
| baggage_id       | BIGINT      | FK -> BAGGAGE.baggage_id; UNIQUE; NOT NULL |
| booking_id       | BIGINT      | FK -> BOOKING.booking_id; NOT NULL         |
| passenger_id     | BIGINT      | FK -> PASSENGER.passenger_id; NOT NULL     |
| check_result     | VARCHAR(30) | NOT NULL                                   |
| created_at       | TIMESTAMP   | NOT NULL                                   |
| updated_at       | TIMESTAMP   | NOT NULL                                   |

#### SECURITY\_CHECK

Stores the results of airport security checks completed by passengers.

| Attribute         | Data type   | Keys and constraints                   |
|-------------------|-------------|----------------------------------------|
| security_check_id | BIGINT      | PK; generated; NOT NULL                |
| passenger_id      | BIGINT      | FK -> PASSENGER.passenger_id; NOT NULL |
| check_result      | VARCHAR(30) | NOT NULL                               |
| created_at        | TIMESTAMP   | NOT NULL                               |
| updated_at        | TIMESTAMP   | NOT NULL                               |

## 4 Relationships and participation constraints

The table states the minimum and maximum number of related records in both directions. The Passenger - Flight many-to-many relationship is resolved through BOOKING.

| Relationship                 | For one first entity   | For one second entity | Explanation                                                                                        |
|------------------------------|------------------------|-----------------------|----------------------------------------------------------------------------------------------------|
| Airline - Flight             | 0..* Flights           | 1 Airline             | An airline may operate no flights or many flights; every flight belongs to one airline.            |
| Airport - Flight (departure) | 0..* departing Flights | 1 departure Airport   | An airport may be the departure point for many flights; every flight has one departure airport.    |
| Airport - Flight (arrival)   | 0..* arriving Flights  | 1 arrival Airport     | An airport may be the arrival point for many flights; every flight has one arrival airport.        |
| Flight - Booking             | 0..* Bookings          | 1 Flight              | A flight may have no bookings or many bookings; every booking refers to one flight.                |
| Passenger - Booking          | 0..* Bookings          | 1 Passenger           | A passenger may make many bookings; every booking belongs to one passenger.                        |
| Booking - BookingChange      | 0..* Changes           | 1 Booking             | A booking may have no changes or many change records; every change belongs to one booking.         |
| Booking - BoardingPass       | 0..1 BoardingPass      | 1 Booking             | A booking may have no boarding pass or one boarding pass; every pass belongs to one booking.       |
| Booking - Baggage            | 0..* Baggage items     | 1 Booking             | A booking may have no baggage or many baggage items; every item belongs to one booking.            |
| Baggage - BaggageCheck       | 0..1 BaggageCheck      | 1 Baggage item        | A baggage item may have no check or one check; every check refers to one baggage item.             |
| Booking - BaggageCheck       | 0..* BaggageChecks     | 1 Booking             | A booking may be referenced by several baggage checks; every baggage check stores one booking.     |
| Passenger - BaggageCheck     | 0..* BaggageChecks     | 1 Passenger           | A passenger may be referenced by several baggage checks; every baggage check stores one passenger. |
| Passenger - SecurityCheck    | 0..* SecurityChecks    | 1 Passenger           | A passenger may complete several security checks; every security check belongs to one passenger.   |

### Cardinality notation

| Notation | Meaning                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------|
| 1        | Exactly one related record is required.                                                                     |
| 0..1     | No related record or one related record is allowed.                                                         |
| 0..*     | No related records or any number of related records is allowed.                                             |
| 1..*     | At least one related record is required.                                                                    |
| 0..0     | Participation is prohibited. It is included in the legend, but no valid relationship in this model uses it. |

## 5 Main constraints

- Every primary key is unique and cannot be NULL.
- Every foreign key must refer to an existing record in its parent table.
- AIRLINE.airline\_code and PASSENGER.passport\_number are unique.
- BOARDING\_PASS.booking\_id is unique, so a booking has at most one boarding pass.
- BAGGAGE\_CHECK.baggage\_id is unique, so a baggage item has at most one baggage check.
- BOOKING.ticket\_price must be greater than or equal to zero, and BAGGAGE.weight\_kg must be greater than zero.
- A flight's arrival airport must differ from its departure airport. Scheduled arrival must be later than scheduled departure.
- BAGGAGE\_CHECK.booking\_id and passenger\_id must match the booking and passenger associated with its BAGGAGE record.

## 6 Conclusion

This work presents an ER model for an international airport database. The model identifies ten entities, shows all primary and foreign keys, assigns a data type to every attribute, and records minimum and maximum participation for each relationship. The result can be used as a basis for implementing the database in a DBMS.